

Université Marie et Louis Pasteur Besançon

Master 2 ISL (Ingénierie Systèmes et Logiciels)

Rapport de projet semestriel

MODELS UNDER CONTROL

*Implémentation de contrôleurs dans des systèmes
cyber-physiques (BIP)*

Auteurs :

DELACROIX Luc
BARTHOD Julien
ROYER Lucas

Encadrantes :

Olga KOUCHNARENKO
Anna GALLONE



Semestres 9 et 10 (Novembre à Février)

24 février 2026

Table des matières

1	Introduction	2
1.1	Contexte du projet	2
1.2	Objectifs	2
2	Synthèse théorique et Analyse	3
2.1	Software Engineering Meets Control Theory	3
2.2	Théorie du Contrôle : Fondamentaux et Stratégies	3
2.3	Outils de collaboration	4
2.3.1	Développement et gestion de version	4
2.3.2	Communication et suivi d'équipe	4
2.3.3	Organisation et centralisation des connaissances	5
2.4	Architecture logicielle (BIP)	5
2.4.1	Le framework générique BIP	5
2.4.2	Application au système Blinky Blocks	6
3	Implémentation des Stratégies de Contrôle	7
3.1	Approche SISO (Single Input Single Output)	7
3.1.1	Architecture des contrôleurs et du Wrapper Générique	7
3.1.2	Le Contrôleur PID	7
3.2	Approche MPC (Model Predictive Control)	9
3.2.1	Modélisation et Principe Théorique	9
3.2.2	Architecture Logicielle C++	10
3.2.3	Implémentation Algorithmique et Gestion des Contraintes	10
3.3	Extension au MIMO (Multi-Input Multi-Output)	11
3.3.1	Généralisation du cadre de contrôle	11
3.3.2	Architecture logicielle du contrôleur MIMO	11
3.3.3	Implémentation du régulateur LQR discret	12
4	Tests, Validation et Résultats	14
4.1	Résultats : Cas PID (SISO)	14
4.1.1	Premier test : environnement C++	14
4.1.2	Deuxième test : applaudimètre modifié	17
4.2	Résultats : Contrôle Prédictif (MPC)	19
4.2.1	Conditions du test	19
4.2.2	Analyse détaillée de la réponse (Tension et Commande)	20
4.3	Résultats : Cas MIMO	21
4.3.1	Mise en place du scénario de test	21
4.3.2	Comportement du système simulé	23
4.3.3	Interprétation générale	23
5	Conclusion et Perspectives	25
5.1	Bilan quantitatif et volume de développement	25
5.2	Difficultés surmontées	25
5.3	Ouvertures et travaux futurs	26
	Remerciements	30

1 Introduction

1.1 Contexte du projet

Ce projet s'inscrit dans le cadre du Master 2 Informatique (ISL) et s'articule autour des problématiques liées aux systèmes cyber-physiques (CPS). Dans ces systèmes, la réalisation de tâches critiques est souvent mise à l'épreuve par des perturbations provenant de l'environnement, pouvant entraîner des écarts entre le comportement attendu et le comportement réel. Pour pallier ces erreurs, nous nous appuyons sur la théorie du contrôle afin d'intégrer des modules de correction, appelés contrôleurs, capables de mesurer l'état du système en continu pour le stabiliser.

Nos travaux se concentrent sur l'utilisation du framework BIP (Behavior Interaction Priority), une architecture rigoureuse permettant la conception de systèmes à base de composants. Ce projet s'intègre plus largement dans le projet de recherche ANR ADAPT, visant à renforcer la fiabilité des systèmes complexes.

1.2 Objectifs

L'objectif principal de notre étude est de doter le framework BIP d'une collection de composants contrôleurs réutilisables, permettant d'intégrer diverses technologies de commande à n'importe quel modèle de conception. Pour ce faire, notre travail se décompose en plusieurs étapes clés :

- Une phase d'exploration de la littérature scientifique pour identifier et comprendre les concepts sous-jacents aux différents types de contrôleurs existants.
- La sélection et l'implantation d'un sous-ensemble de ces contrôleurs en langage BIP/C++.
- La validation de ces implantations à travers des simulations de modèles de systèmes cyber-physiques, afin de vérifier l'efficacité de la correction apportée face aux perturbations.
- La création d'une documentation claire expliquant le fonctionnement de la librairie développée.

2 Synthèse théorique et Analyse

2.1 Software Engineering Meets Control Theory

Le document *Software Engineering Meets Control Theory* (Filieri et al., 2015)[1] pose les fondations théoriques justifiant l'approche de notre projet. Les auteurs y dressent un constat majeur : les systèmes logiciels modernes, en particulier les systèmes cyber-physiques (CPS), évoluent dans des environnements hautement dynamiques et incertains. Les approches d'ingénierie logicielle traditionnelles peinent à garantir la fiabilité de ces systèmes face à des perturbations imprévues, des pannes matérielles ou des variations de charge.

Pour répondre à ce défi, l'article propose de traiter les logiciels adaptatifs sous le prisme de l'automatique (la théorie du contrôle). En considérant le système logiciel comme un procédé physique (le "Plant"), il devient possible de lui appliquer une boucle de rétroaction (Feedback Loop). L'état du système est mesuré en continu, comparé à une cible idéale, et un contrôleur ajuste dynamiquement les paramètres pour corriger l'écart (voir figure 1).

L'apport fondamental de cette littérature pour notre projet est de démontrer que l'intégration de contrôleurs mathématiques au sein de l'architecture logicielle permet d'obtenir des garanties formelles sur le comportement du système, telles que la stabilité, la précision ou encore le temps d'établissement, et ce, malgré la présence de perturbations externes. C'est précisément cette philosophie que nous cherchons à appliquer en dotant le framework BIP de composants de contrôle rigoureux.

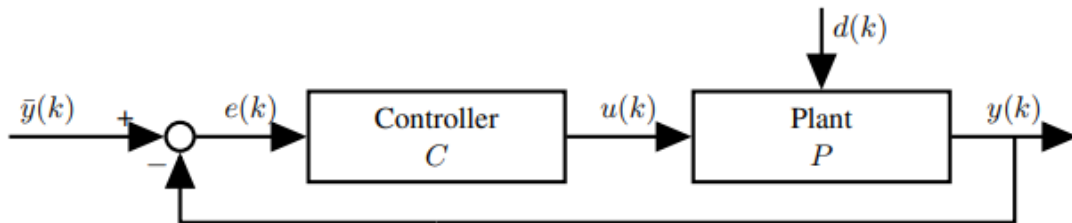


FIGURE 1 – Architecture d'une boucle de rétroaction appliquée au logiciel

2.2 Théorie du Contrôle : Fondamentaux et Stratégies

La mise en œuvre de notre bibliothèque s'appuie sur les principes de conception des systèmes de contrôle, largement documentés dans la littérature classique telle que *Control System Design* (Goodwin et al., 2001)[2]. Dans le cadre de ce projet, nous nous sommes focalisés sur des régulateurs en temps discret, adaptés à une exécution algorithmique.

La théorie du contrôle distingue généralement deux grandes familles de systèmes que nous avons explorés :

- **SISO (Single Input Single Output)** : Ces systèmes possèdent une unique variable d'action pour contrôler une unique variable de sortie. Pour ces systèmes, nous avons étudié et implémenté le contrôleur PID (Proportionnel-Intégral-Dérivé). C'est le standard industriel par excellence. Il est dit "réactif" car il calcule sa com-

mande en se basant sur l'erreur présente (action proportionnelle), l'accumulation des erreurs passées (action intégrale) et la tendance de l'erreur (action dérivée).

- **MIMO (Multi-Input Multi-Output)** : Ces systèmes font intervenir de multiples variables interconnectées, où une action influe sur plusieurs sorties simultanément. L'utilisation de multiples PID indépendants y est souvent inefficace à cause du couplage.

Pour aller plus loin que les limites réactives du PID, la littérature met en avant les contrôleurs basés sur un modèle. C'est cette approche que nous avons adoptée pour nos contrôleurs MPC (Model Predictive Control) et LQR (Linear Quadratic Regulator) :

- **L'anticipation** : Ces algorithmes intègrent une représentation mathématique (modèle d'état) de la physique du système. Ils ne se contentent plus de réagir à une erreur, mais prédisent l'évolution temporelle future du procédé.
- **L'optimisation sous contrainte** : La commande appliquée n'est plus une simple équation proportionnelle, mais le résultat d'une minimisation d'une fonction de coût (trouver le meilleur compromis entre la précision demandée et l'effort énergétique de l'actionneur). L'approche MPC permet en outre de garantir de manière stricte le respect de contraintes de sécurité.

Ces bases théoriques nous ont permis de définir un périmètre d'implémentation clair : partir d'un régulateur empirique et universel (le PID) pour évoluer vers des régulateurs matriciels prédictifs (MPC, LQR) capables de gérer la complexité et les contraintes strictes des systèmes cyber-physiques modernes.

2.3 Outils de collaboration

La réussite de ce projet repose sur une coordination efficace entre les membres de l'équipe et un suivi rigoureux des avancées. Pour ce faire, nous avons mis en place un écosystème d'outils numériques structuré autour de trois axes : le développement, la communication et la gestion des connaissances.

2.3.1 Développement et gestion de version

Initialement, notre flux de travail s'appuyait sur GitLab. Cependant, suite à des contraintes techniques, nous avons opéré une migration vers GitHub. Cette plateforme nous permet de centraliser notre code source, de gérer les versions de manière collaborative et d'assurer l'intégrité des développements en BIP/C++.

2.3.2 Communication et suivi d'équipe

La communication est centralisée sur Discord. Cet outil nous permet d'échanger quotidiennement par messages et d'organiser nos réunions internes hebdomadaires. Ces sessions sont essentielles pour :

- Faire le point sur le travail accompli durant la semaine passée ;
- Identifier et résoudre collectivement les problèmes techniques rencontrés ;
- Partager nos découvertes issues de l'exploration de la littérature.

En complément, des réunions régulières avec les responsables du projet (Mme Kouchnarenko et Mme Gallone) sont organisées pour valider l'état d'avancement global et obtenir des réponses à nos questions d'ordre théorique ou méthodologique.

2.3.3 Organisation et centralisation des connaissances

Enfin, nous utilisons Notion comme tour de contrôle du projet. Cet outil remplit plusieurs fonctions critiques :

- **Gestion des tâches** : Suivi de la répartition du travail entre les membres du groupe.
- **Historique et traçabilité** : Rédaction systématique de comptes-rendus pour chaque réunion (internes ou avec les encadrantes).
- **Base de connaissances** : Centralisation des découvertes, explications techniques et points clés du projet. Cette documentation partagée permet à chaque membre de l'équipe d'assimiler rapidement des concepts qu'il n'aurait pas encore abordés individuellement, garantissant ainsi une montée en compétences homogène du groupe.

2.4 Architecture logicielle (BIP)

Le projet s'appuie sur le framework BIP (*Behavior, Interaction, Priority*)[3], une approche logicielle rigoureuse basée sur des composants, conçue pour la construction de systèmes complexes temporels et réactifs. Ce framework nous permet de séparer formellement l'architecture du système de son code algorithmique.

2.4.1 Le framework générique BIP

Dans la méthodologie BIP, tout système est modélisé selon trois couches fondamentales :

1. **Behavior (Le Comportement - Atomes) :**

L'architecture repose sur des briques de base appelées *Atomes*. Un atome représente un composant logiciel ou matériel autonome. Son comportement interne est défini rigoureusement par un automate à états finis (transitions et états) couplé à des variables locales (code C++). Un atome expose des "ports" qui sont ses seuls points de contact avec l'extérieur.

2. **Interaction (Les Connecteurs) :**

Les atomes ne communiquent jamais directement entre eux de manière asynchrone ou par appels de fonctions classiques. Ils utilisent des *Connecteurs*. Un connecteur relie les ports de plusieurs atomes pour définir une interaction. Il assure la synchronisation temporelle entre les composants et le transfert sécurisé des données.

3. **Priority (Les Priorités) :**

Lorsque plusieurs interactions sont possibles simultanément dans le système, la couche de priorités permet d'édicter des règles pour lever les ambiguïtés. Cela garantit un comportement d'exécution parfaitement déterministe et sans interblocage (*deadlock*).

2.4.2 Application au système Blinky Blocks

Bien que l'approche BIP et notre bibliothèque de contrôleurs soient conçues pour être génériques et applicables à n'importe quel système cyber-physique (CPS), le sujet de notre projet était fortement orienté vers l'étude des modules *Blinky Blocks*. Ces blocs modulaires constituant notre cas d'usage principal (notamment en lien avec le projet ADAPT), nous avons naturellement choisi de structurer notre application et nos exemples autour de ce matériel spécifique.

Dans ce cadre, nous avons exploité cette séparation des préoccupations offerte par BIP pour isoler la physique de la logique de contrôle. Le système est modélisé par deux types d'atomes principaux interagissant dans une boucle fermée :

- **L'Atome "Blinky Block" :**

Il modélise le composant matériel (le Plant). Cet atome simule la physique du système. Il expose des ports de sortie pour envoyer l'état de ses capteurs et des ports d'entrée pour recevoir les actions correctives.

- **L'Atome "Contrôleur" :**

Il encapsule notre bibliothèque d'algorithmes. Son automate interne est simple : il attend une donnée sur son port d'entrée de mesure, déclenche le calcul de correction via la méthode C++ `compute()`, puis expose le résultat sur son port de sortie de commande.

Ces deux atomes sont reliés par des connecteurs de rétroaction. Un premier connecteur lie le port "Sortie Tension" du Blinky Block au port "Entrée Mesure" du Contrôleur, tandis qu'un second ferme la boucle en liant la commande du contrôleur à l'actionneur du Blinky Block.

Cette architecture modulaire forte est un atout majeur : elle nous permet de développer, tester et compiler nos contrôleurs mathématiques en C++ de manière totalement isolée, puis de les "plugger" dynamiquement dans l'atome Contrôleur sans jamais avoir à modifier le code de l'automate du Blinky Block.

3 Implémentation des Stratégies de Contrôle

Cette section détaille le développement incrémental des contrôleurs, allant du cas simple (SISO) aux cas complexes (MIMO).

3.1 Approche SISO (Single Input Single Output)

3.1.1 Architecture des contrôleurs et du Wrapper Générique

Dans le but d'uniformiser nos futurs contrôleurs SISO, nous avons mis en place deux interfaces distinctes :

- `IControllerSISO.hpp` : définissant le prototypage des contrôleurs SISO ;
- `ControllerBIPInterface.hpp` : définissant une liste de wrappers utilisables dans un programme BIP.

Interface générique des contrôleurs Les avantages d'implémenter une interface unique pour tous les contrôleurs SISO sont multiples.

Tout d'abord, cela permet de créer un type générique qui englobe tous les contrôleurs, ce qui facilite son utilisation dans BIP. Ensuite, cela assure une certaine cohérence dans la structure des contrôleurs, ce qui permet non seulement de créer des wrappers communs (ce qu'on verra après), mais aussi de plus facilement enrichir les contrôleurs existants à l'avenir. Enfin, dans l'optique d'un enrichissement de cette bibliothèque, cela permettra aux futurs développeurs de créer de nouveaux contrôleurs plus facilement grâce à une structure commune et des wrappers déjà existants.

Interface de wrappers pour BIP Pour ce qui est de la deuxième interface, elle permet de définir une liste de wrappers pour les contrôleurs SISO. Cette interface est indispensable à l'utilisation des contrôleurs dans un programme BIP, car nous avons fait le choix de développer nos contrôleurs avec une syntaxe objet (nous permettant d'utiliser au maximum les possibilités offertes par C++).

Or, il est impossible d'utiliser directement un objet C++ dans un programme BIP ; nous sommes donc obligés de passer par un intermédiaire sous la forme fonctionnelle.

Dans cette interface sont donc définis des wrappers permettant :

- d'exécuter la méthode `compute` des contrôleurs ;
- de réinitialiser les valeurs d'un contrôleur ;
- de détruire un contrôleur ;
- d'accéder aux getters d'un contrôleur.

À noter que les wrappers permettant de créer un contrôleur sont implémentés directement dans la même interface que leur contrôleur respectif (cela permet d'éviter des problèmes d'import et de compilation dans les programmes BIP).

3.1.2 Le Contrôleur PID

Le contrôleur PID (Proportional-Integral-Derivative) implémenté est destiné à la régulation de systèmes SISO (Single Input Single Output). Il repose sur le calcul classique de la correction à partir de l'erreur entre la consigne et la mesure du système.

Définition de l'erreur À chaque appel de la méthode `compute(dt)`, l'erreur instantanée est calculée comme :

$$e(t) = y_{\text{ref}}(t) - y(t) \quad (1)$$

où y_{ref} représente la valeur cible (consigne) et y la valeur mesurée.

Terme proportionnel Le terme proportionnel est directement proportionnel à l'erreur :

$$P(t) = K_p e(t) \quad (2)$$

Il permet une réaction immédiate à l'écart courant.

Terme intégral Le terme intégral correspond à l'accumulation temporelle de l'erreur :

$$I(t) = K_i \int_0^t e(\tau) d\tau \quad (3)$$

Dans l'implémentation discrète (pas de temps dt), l'intégrale est approximée par :

$$I_k = I_{k-1} + e_k dt \quad (4)$$

Ce terme permet d'annuler l'erreur statique en régime permanent.

Terme dérivé Le terme dérivé anticipe l'évolution de l'erreur en calculant sa variation :

$$D(t) = K_d \frac{de(t)}{dt} \quad (5)$$

En discret :

$$D_k = K_d \frac{e_k - e_{k-1}}{dt} \quad (6)$$

Il agit comme un amortissement dynamique en limitant les oscillations et le dépassement.

Calcul de la commande La correction appliquée au système est la somme des trois contributions :

$$u(t) = P(t) + I(t) + D(t) \quad (7)$$

Dans l'implémentation, cette valeur peut être limitée par des bornes minimales et maximales :

$$u_{\text{sat}} = \text{clamp}(u, u_{\text{min}}, u_{\text{max}}) \quad (8)$$

Ce mécanisme permet de modéliser la saturation d'un actionneur réel.

Mémoire interne Le contrôleur conserve en mémoire :

- l'intégrale accumulée,
- l'erreur précédente (nécessaire au calcul dérivé),
- la dernière correction calculée.

Une méthode `reset()` permet de réinitialiser ces états internes.

Résumé Cette implémentation correspond à un PID discret de type parallèle :

$$u_k = K_p e_k + K_i \sum e_k dt + K_d \frac{e_k - e_{k-1}}{dt} \quad (9)$$

Elle constitue une structure standard de régulation numérique, adaptée aux simulations et aux systèmes embarqués.

3.2 Approche MPC (Model Predictive Control)

Cette section détaille la conception et l'implémentation du contrôleur prédictif. Contrairement au PID qui réagit uniquement à l'erreur passée, le MPC utilise un modèle interne pour anticiper le comportement futur du système et adapter la commande en conséquence.

3.2.1 Modélisation et Principe Théorique

Le contrôleur repose sur une modélisation mathématique du système (Espace d'État) adaptée à notre cas simple (SISO).

Nous utilisons une équation de récurrence pour prédire l'évolution temporelle :

$$y_{k+1} = a \cdot y_k + b \cdot u_k \quad (10)$$

De manière générale, cela signifie que la valeur future de la sortie (y_{k+1}) dépend de la dynamique propre du système à l'instant actuel ($a \cdot y_k$) et de l'action de commande appliquée par le contrôleur ($b \cdot u_k$).

L'objectif de l'algorithme est de trouver la séquence de commande optimale en minimisant une fonction de coût (score de performance J) sur un horizon de prédiction de N pas :

$$J = \sum_{k=1}^N [q \cdot (Cible - y_k)^2 + r \cdot u_k^2] \quad (11)$$

où Cible représente la valeur idéale que l'on souhaite que le système atteigne et maintienne.

Cette formule mathématique recherche le meilleur compromis possible entre deux objectifs contradictoires :

- Minimiser l'erreur de suivi (rester le plus proche possible de la cible, modulé par le poids q).
- Minimiser l'effort de commande (ne pas solliciter excessivement les actionneurs, modulé par le poids r).

3.2.2 Architecture Logicielle C++

Le code est organisé en trois parties pour garantir la modularité et la généricité du composant.

1. **L'Interface (IControllerMPC)** : Elle liste les méthodes obligatoires pour configurer le modèle physique, définir les contraintes de sécurité et calculer la commande, indépendamment du système ciblé.
2. **Le Cœur Algorithmique (MPCController)** : C'est la classe concrète qui exécute les calculs mathématiques et la boucle de prédiction glissante.
3. **Le Wrapper (ControllerBIPInterface)** : Cette couche simplifie l'intégration du contrôleur générique dans l'environnement spécifique de notre application (BIP) en gérant l'instanciation et le paramétrage par défaut.

3.2.3 Implémentation Algorithmique et Gestion des Contraintes

Dans une approche classique, le calcul de la commande du MPC nécessite de résoudre un problème d'optimisation sous contraintes à chaque pas de temps, ce qui requiert généralement l'utilisation de solveurs de programmation quadratique. Bien que très puissants, ces outils génériques présentent plusieurs inconvénients dans notre contexte : ils introduisent une dépendance logicielle importante, leur empreinte mémoire est souvent trop élevée pour des systèmes embarqués (comme des Blinky Blocks), et leur nature itérative rend le temps d'exécution variable, compliquant ainsi les garanties de fonctionnement en temps réel.

Pour pallier cela et tirer parti de la simplicité de notre modèle, nous avons écarté l'utilisation de ces bibliothèques au profit d'une méthode de résolution analytique directe, garantissant un temps de calcul minimal et déterministe.

À chaque cycle de calcul, l'algorithme exécute la séquence suivante :

— **Étape 1 : Prédiction Idéale**

Le contrôleur simule l'évolution du système sur N pas de temps futurs. Il calcule la commande théorique optimale pour atteindre la cible, sans se soucier des limites physiques dans un premier temps.

— **Étape 2 : Vérification de Sécurité (Safety Check)**

L'algorithme vérifie si l'application de cette commande théorique risque de faire franchir au système une limite critique de sécurité (ex : surchauffe, tension trop basse). Si la prédiction détecte une violation imminente, le contrôleur calcule une "commande de sauvetage" :

$$u_{sauvetage} = \frac{Limite - a \cdot y_k}{b} \quad (12)$$

Cette action préventive force le système à se stabiliser exactement sur la limite autorisée au lieu de la franchir.

— **Étape 3 : Saturation de la Commande (Limites Physiques)**

Enfin, la commande calculée est adaptée pour s'assurer qu'elle respecte les limites physiques des actionneurs du système (un actionneur ne pouvant pas délivrer une puissance infinie).

Grâce à cette méthode d’horizon glissant, le contrôleur est capable d’anticiper les perturbations et les contraintes, agissant de manière proactive pour garantir à la fois la performance et la sécurité du système cyber-physique contrôlé.

3.3 Extension au MIMO (Multi-Input Multi-Output)

Cette section présente l’extension du cadre de contrôle vers des systèmes multi-entrées multi-sorties. Contrairement à une approche où l’on mettrait plusieurs contrôleurs PID indépendant côte-à-côte, le contrôleur MIMO prend en compte le couplage entre les variables : une seule action peut modifier plusieurs sorties à la fois. Le contrôleur calcule donc une réponse globale coordonnée.

3.3.1 Généralisation du cadre de contrôle

Dans l’approche SISO, le système est représenté par une relation entre une seule entrée et une seule sortie. Chaque boucle de régulation peut être conçue indépendamment, ce qui simplifie l’analyse et le réglage du contrôleur. Cependant, cette hypothèse représente une limitation lorsque les variables du système sont reliées dynamiquement.

Dans le cas d’un système MIMO, plusieurs actionneurs agissent en même temps sur diverses variables d’état. Il devient donc essentiel de privilégier une représentation globale du système, généralement sous la forme d’un modèle d’état discret :

$$x_{k+1} = Ax_k + Bu_k \quad (13)$$

où :

- $x \in \mathbb{R}^n$ représente le vecteur d’états,
- $u \in \mathbb{R}^m$ le vecteur des entrées,
- A et B sont les matrices décrivant la dynamique du système.

Cette représentation matricielle permet de prendre en compte les interactions entre variables : une action sur une entrée peut influencer plusieurs sorties. La commande est alors calculée globalement par retour d’état selon :

$$u_k = -Kx_k \quad (14)$$

où K est une matrice de gain déterminée à partir du modèle du système.

Cette approche permet ainsi de traiter le système dans sa globalité, en tenant compte des interactions entre les différentes variables d’état.

3.3.2 Architecture logicielle du contrôleur MIMO

Afin d’assurer une structure claire et réutilisable, une interface générique de contrôleur MIMO a été définie. Cette interface fixe le cadre commun à tout contrôleur multivariable et permet d’uniformiser son utilisation afin d’en standardiser l’utilisation.

Elle définit notamment :

- les dimensions du système (n états, m entrées)
- le stockage de l’état courant

- la gestion de la consigne
- une méthode de calcul de la commande appelée à chaque pas d'échantillonnage

Cette organisation permet de séparer clairement la structure générale du contrôleur de son implémentation spécifique. La classe concrète développée pour ce projet implémente cette interface en intégrant le modèle discret du système via les matrices A et B , ainsi que les paramètres nécessaires au calcul de la commande.

Le contrôleur conserve en interne :

- les matrices du modèle discret
- les matrices de pondération Q et R
- la matrice de gain K calculée automatiquement
- l'état courant utilisé pour le calcul de la commande

Dans le cas d'un système linéaire, les matrices A , B , Q et R étant constantes, la solution de l'équation de Riccati converge vers une matrice stationnaire. Le gain optimal K est donc constant.

Il n'est alors recalculé que si le modèle ou les matrices de pondération sont modifiées. Ce choix repose sur une propriété du LQR et non sur une simple optimisation pratique.

Cette organisation garantit une implémentation claire, extensible et facilement adaptable à différents systèmes MIMO.

3.3.3 Implémentation du régulateur LQR discret

Le contrôleur MIMO développé repose sur une stratégie de retour d'état de type LQR (Linear Quadratic Regulator) en temps discret. Cette méthode permet de déterminer automatiquement une matrice de gain optimale à partir du modèle du système.

Le système est décrit par le modèle discret :

$$x_{k+1} = Ax_k + Bu_k \quad (15)$$

Le principe du LQR consiste à minimiser un critère quadratique de la forme :

$$J = \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k) \quad (16)$$

où la matrice Q pondère l'importance accordée aux écarts des états et la matrice R pénalise l'effort de commande.

À partir des matrices A , B , Q et R , la matrice de gain K est obtenue par la résolution numérique de l'équation de Riccati discrète. La matrice P solution de cette équation permet de calculer le gain selon :

$$K = (R + B^T P B)^{-1} B^T P A \quad (17)$$

Le gain est calculé une seule fois lors de l'initialisation ou lorsque les paramètres du modèle sont modifiés.

La loi de commande appliquée à chaque pas d'échantillonnage est alors :

$$u_k = -Kx_k \quad (18)$$

La commande est ainsi déterminée par une simple opération matricielle, ce qui permet une exécution efficace à chaque itération. Le réglage du comportement dynamique du système s'effectue directement via le choix des matrices Q et R , qui définissent le compromis entre rapidité de convergence et amplitude de la commande.

4 Tests, Validation et Résultats

4.1 Résultats : Cas PID (SISO)

Dans cette section seront détaillés les deux scénarios mis en place pour tester le contrôleur PID.

4.1.1 Premier test : environnement C++

Le premier test que nous avons mis en place est un test uniquement en C++. Afin d'évaluer les performances du contrôleur PID dans des conditions réalistes, un banc d'essai numérique a été mis en place. Celui-ci repose sur la simulation d'un système du premier ordre soumis à du bruit de mesure et à des perturbations externes variables dans le temps. Nous avons également appliqué le même protocole de test aux contrôleurs P et PI.

Modèle du procédé Le système simulé est un processus linéaire du premier ordre décrit par l'équation différentielle :

$$\dot{y}(t) = -a y(t) + b u(t) + d(t) \quad (19)$$

où :

- $y(t)$ représente la sortie du système,
- $u(t)$ la commande appliquée,
- a la constante de dissipation,
- b le gain du procédé,
- $d(t)$ une perturbation externe.

La discrétisation est réalisée par une méthode d'Euler explicite avec un pas de simulation $dt = 0,1 \text{ s}$:

$$y_{k+1} = y_k + dt (-a y_k + b u_k + d_k) \quad (20)$$

Les paramètres choisis sont $a = 0,8$ et $b = 1,0$.

Bruit de mesure Afin de simuler un capteur imparfait, un bruit uniforme est ajouté à la sortie réelle :

$$y_{\text{mes}} = y + \eta \quad (21)$$

où η est une variable aléatoire uniforme bornée par une amplitude fixée à 1.0. Le contrôleur agit donc sur une mesure bruitée et non sur la valeur réelle du système.

Structure du contrôleur Le contrôleur PID est paramétré avec les gains :

$$K_p = 0,6, \quad K_i = 0,1, \quad K_d = 0,05 \quad (22)$$

La commande est limitée entre 0 et 255, ce qui permet de modéliser la saturation d'un actionneur réel.

Scénario de test La simulation s'étend sur une durée totale de 100 s. Le scénario se déroule en trois phases :

1. **Réponse à échelon** : à $t = 1$ s, la consigne passe de 0 à 100.
2. **Perturbation modérée** : à $t = 40$ s, une perturbation constante de valeur 5 est appliquée.
3. **Perturbation renforcée** : à $t = 70$ s, la perturbation est augmentée à 8.

Ce protocole permet d'évaluer :

- la rapidité de réponse du contrôleur,
- sa capacité à rejeter une perturbation constante,
- son comportement en présence de bruit,
- sa stabilité sous saturation.

Enregistrement des résultats À chaque pas de simulation sont enregistrés :

- le temps t ,
- la consigne,
- la sortie réelle du système,
- la commande appliquée.

Ces données sont exportées au format CSV afin de les représenter sous forme de graphiques.

Objectif de l'expérimentation Ce test vise à analyser la robustesse du contrôleur PID face aux incertitudes (bruit de mesure) et aux perturbations dynamiques. Il permet également d'observer l'influence du terme dérivé dans un contexte bruité ainsi que l'efficacité du terme intégral dans le rejet des perturbations constantes. De plus, ce test a aussi permis de tester un aspect plus structurel de notre contrôleur. En effet, nous avons pu constater que notre implémentation de PID ainsi que de notre bibliothèque en général était pertinente et fonctionnelle.

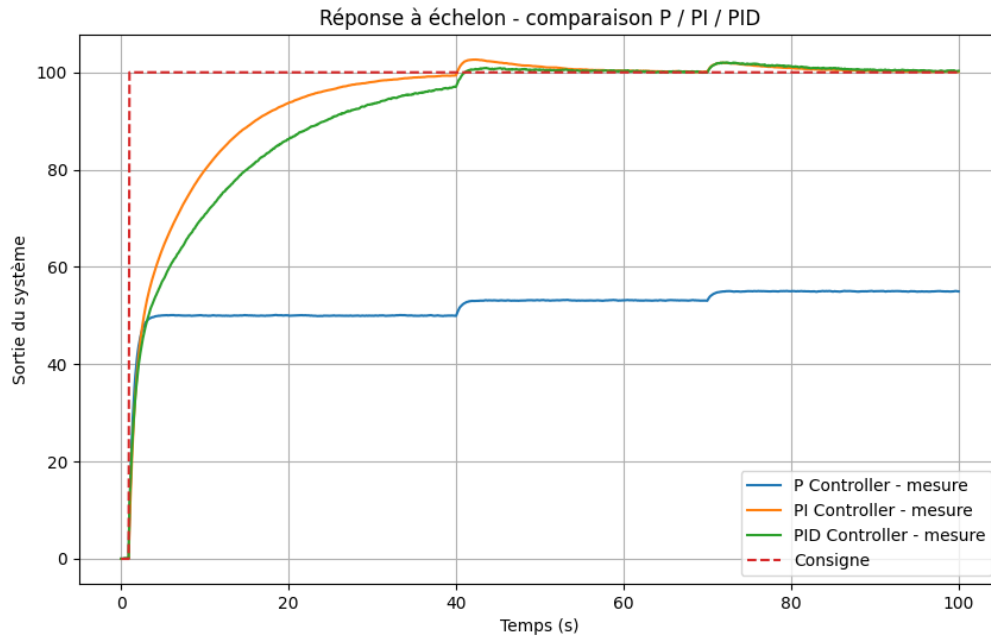


FIGURE 2 – Mesures transmises aux contrôleurs

Le graphique de la figure 2 montre les différentes mesures réalisées par le système tout au long de la simulation. On peut constater que dans un premier temps très court, la consigne est à 0 et ensuite passe instantanément à 100. Cela provoque une forte hausse des mesures pour les trois contrôleurs qui tend à se stabiliser. Une fois les valeurs stabilisées, on remarque quelques petits sursauts, cela est dû aux perturbations dans le système. L'intérêt de ce test est de se rendre compte qu'en-dehors du contrôleur P qui est stabilisé loin de la valeur cible, il n'y a pas de grandes différences d'efficacité entre PI et PID à condition que ces derniers soient correctement paramétrés.

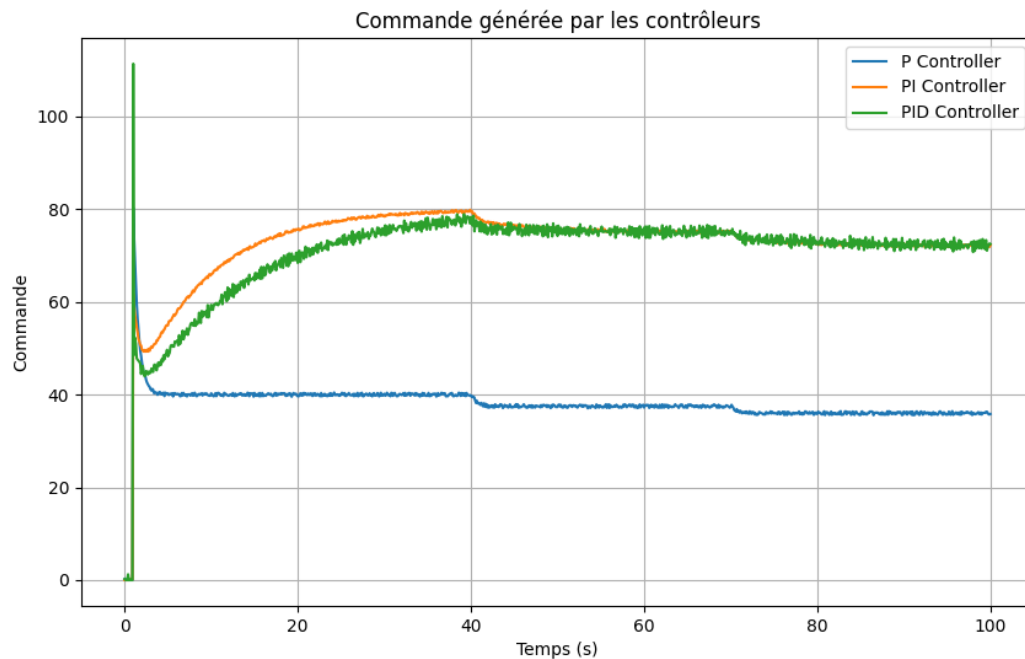


FIGURE 3 – Commandes générées par les contrôleurs

Le graphique de la figure 3 montre quant à lui les différentes commandes générées par les contrôleurs. On constate que les commandes générées suivent la même dynamique que les mesures qui leur sont associées. Toutefois, on peut observer un léger bruit sur les courbes qui est lié à la perturbation, les contrôleurs essayant constamment de corriger les perturbations minimales.

4.1.2 Deuxième test : applaudimètre modifié

Cette section détaille les résultats d'un test du contrôleur PID intégré dans un programme BIP. Nous avons repris le concept de l'applaudimètre, bien que celui-ci ait été largement remanié afin de créer un scénario plus dynamique et plus représentatif d'un comportement réel.

Fonctionnement de l'applaudimètre Dans cette nouvelle version, le comportement diffère sensiblement du premier test.

Le système fonctionne selon la chaîne de rétroaction suivante :

1. Un automate génère un signal sonore en fonction de la commande qui lui est envoyée (initialement nulle).
2. Ce signal est transmis à un second automate représentant le microphone du module *BlinkyBlocks*.
3. Le son capté est ensuite envoyé au MCU, lequel appelle le contrôleur PID et enregistre les données dans les logs.
4. la commande générée par le contrôleur PID est ensuite renvoyée à l'automate qui génère le signal sonore (étape 1).

Pour mieux visualiser la structure du test, on peut établir l'analogie suivante :

- L'automate générant le son représente le public d'une salle.
- Le contrôleur PID joue le rôle du chauffeur de salle, ajustant la dynamique des applaudissements afin d'atteindre un niveau cible.

Durant toute la simulation, une couleur RGB est générée et enregistrée dans les logs en fonction de l'intensité des applaudissements. Un code couleur est appliqué selon la valeur mesurée.

La consigne du contrôleur étant fixée à 100, le test inclut :

- des variations mineures constantes (bruit),
- des perturbations plus importantes à intervalles réguliers.

Ces perturbations sont essentielles pour observer le comportement dynamique du PID. Sans elles, l'intensité convergerait vers 100 et resterait stable, ce qui ne représenterait pas une situation réaliste.

Résultats du test Les couleurs des LEDs sont déterminées selon l'intensité des applaudissements :

- Si l'intensité est inférieure à 50, seule la LED bleue est activée.
- Si l'intensité est égale à 100 (valeur cible), la LED est verte.
- Si l'intensité est comprise entre 50 et 100, la couleur évolue progressivement du bleu vers le vert selon la proximité avec 100.
- Si l'intensité dépasse 100, la couleur tend progressivement vers le rouge.

Cette représentation visuelle permet d'observer intuitivement la performance du contrôleur : une LED majoritairement verte indique un bon suivi de consigne, tandis que des dérives vers le bleu ou le rouge traduisent respectivement un déficit ou un excès d'intensité. Ce qu'il faut comprendre dans la figure 4 est le PID génère des commandes dans le but de stabiliser l'intensité des applaudissements et tendre vers la consigne de 100 (droite pointillée verte). Les grands sursauts sont des perturbations arbitraires ajoutées à la mesure du premier automate à un intervalle régulier dans le but de tester les performances du PID. Pour ce qui est des plus petites perturbations, elles sont générées aléatoirement à chaque mesure et représentent un bruit imprévisible.

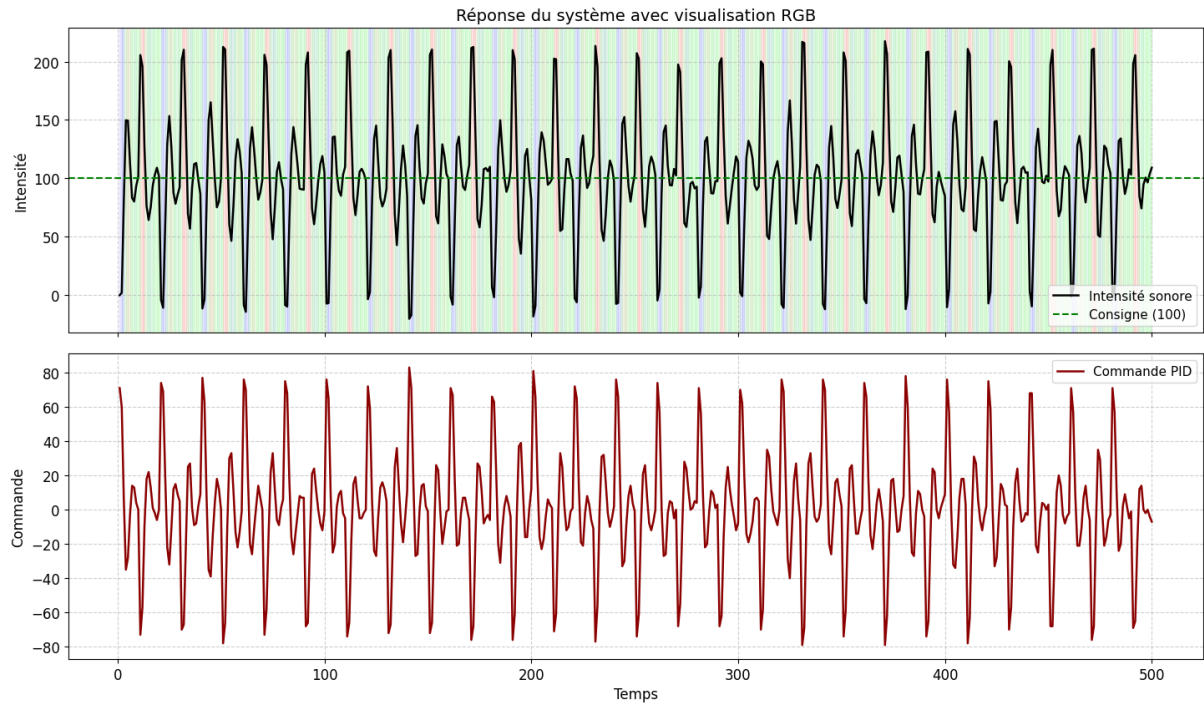


FIGURE 4 – Résultats des tests PID BIP

4.2 Résultats : Contrôle Prédictif (MPC)

Cette section détaille l'évaluation du contrôleur MPC face à un scénario de perturbation sévère, mettant à l'épreuve sa capacité à maintenir une consigne tout en respectant des contraintes de sécurité strictes.

4.2.1 Conditions du test

Le système simulé (modélisant par exemple l'alimentation d'un module Blinky Block) est soumis aux conditions suivantes :

- **Cible (Target)** : Maintien d'une tension de $5.0V$.
- **Sécurité physique** : La tension ne doit sous aucun prétexte descendre en dessous de $3.5V$.
- **Perturbation** : Entre les instants $t = 30$ et $t = 60$, une forte charge imprévue (fuite de tension de $0.5V$ par pas de temps) est appliquée au système.
- **Configuration MPC** : Pour anticiper l'erreur statique liée à cette perturbation non modélisée, une marge de sécurité a été configurée dans le contrôleur, fixant sa limite interne à $4.0V$.

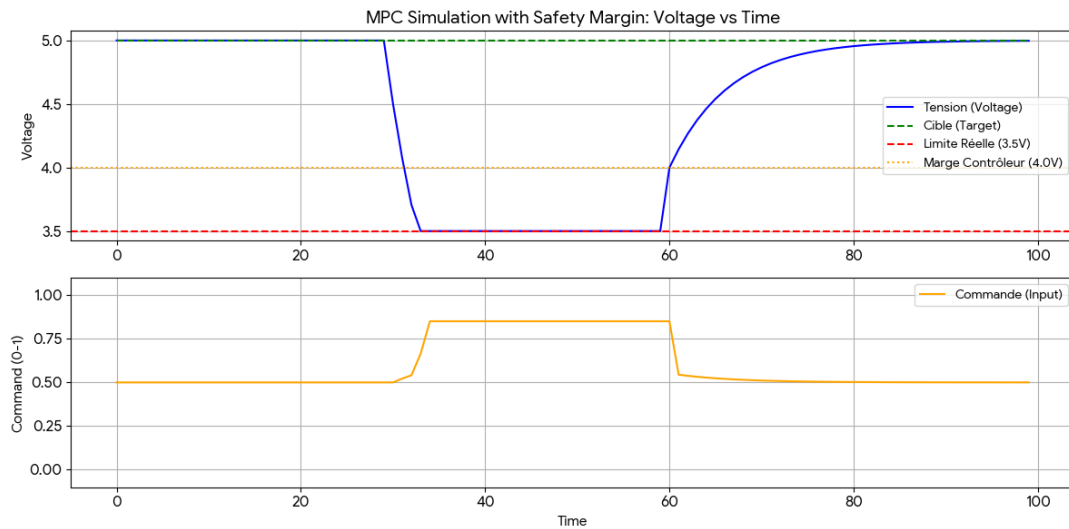


FIGURE 5 – Réponse du contrôleur MPC avec marge de sécurité face à une perturbation

4.2.2 Analyse détaillée de la réponse (Tension et Commande)

L'observation du graphique (Figure 5) permet de distinguer trois phases distinctes dans le comportement du contrôleur :

1. **Phase 1 - Régime permanent** ($t = 0$ à $t = 29$) :

Le contrôleur démontre une excellente compréhension du modèle physique. La commande (courbe orange) se stabilise rapidement autour de 0.5 (50% de la puissance), ce qui permet à la tension de se maintenir de manière parfaitement stable à 5.0V (le maximum atteint sur la courbe), sans aucune oscillation ni dépassement (overshoot).

2. **Phase 2 - Rejet de perturbation et "Mode Survie"** ($t = 30$ à $t = 60$) :

Dès l'apparition de la perturbation, la tension chute brutalement. Le contrôleur réagit en augmentant la commande jusqu'à environ 0.85 (85% d'effort).

- Le Minimum atteint : La tension descend jusqu'à un minimum de 3.5V et s'y stabilise parfaitement.
- Explication du palier : Le MPC fonctionne sur le principe du moindre effort sous contrainte. L'algorithme calcule qu'il serait trop coûteux (ou physiquement impossible) de remonter à 5.0V pendant cette crise. Il choisit donc d'appliquer la commande strictement nécessaire pour ne pas franchir sa limite interne (4.0V).
- Succès de la marge : La perturbation non mesurée tirant le système 0.5V plus bas que prévu, le système atterrit physiquement et précisément sur la vraie limite critique de 3.5V. La contrainte matérielle est ainsi parfaitement respectée sans saturer inutilement le moteur à 100%.

3. **Phase 3 - Récupération** ($t > 60$) :

Dès que la perturbation disparaît, le contrôleur relâche immédiatement son effort de sauvetage. La tension remonte rapidement et proprement vers la consigne cible de 5.0V.

Conclusion du test MPC : Ces résultats valident l'implémentation de l'algorithme "Receding Horizon". Contrairement à un contrôleur classique qui aurait tenté de lutter

aveuglement contre la chute (risquant une saturation instable de la commande), le MPC a fait preuve de robustesse en gérant intelligemment le compromis entre le suivi de la consigne et le respect absolu de la contrainte de sécurité.

4.3 Résultats : Cas MIMO

4.3.1 Mise en place du scénario de test

Afin d'évaluer le comportement du contrôleur LQR MIMO, une simulation a été réalisée sur un modèle discret simplifié de drone évoluant dans un plan 2D.

Le système est décrit par un vecteur d'état composé des positions et vitesses :

$$x = [p_x, p_y, v_x, v_y]$$

Les entrées correspondent aux forces appliquées :

$$u = [u_x, u_y]$$

L'objectif du test est d'atteindre et de maintenir une position cible. Les matrices de pondération Q et R ont été choisies de manière à accorder une importance plus forte aux erreurs de position qu'aux vitesses, tout en limitant l'impact des forces appliquées.

Deux scénarios ont été simulés : un fonctionnement nominal sans perturbation, et un second cas dans lequel une perturbation externe, simulant du vent, vient dévier temporairement le système. Dans les deux cas, le drone est initialement placé en position $(0, 1)$ avec une vitesse nulle et doit atteindre la cible $(1, 2)$ avec une vitesse finale nulle.

Les résultats sont visualisés à l'aide d'un script Python développé spécifiquement dans le cadre de ce projet. Celui-ci génère trois graphiques distincts : l'évolution des positions (Figure 6), des vitesses (Figure 7) et des forces appliquées (Figure 8). Pour chaque graphique, l'axe horizontal représente le temps, et l'axe vertical la grandeur correspondante. Les courbes comparent les composantes selon x et y , avec et sans perturbation.

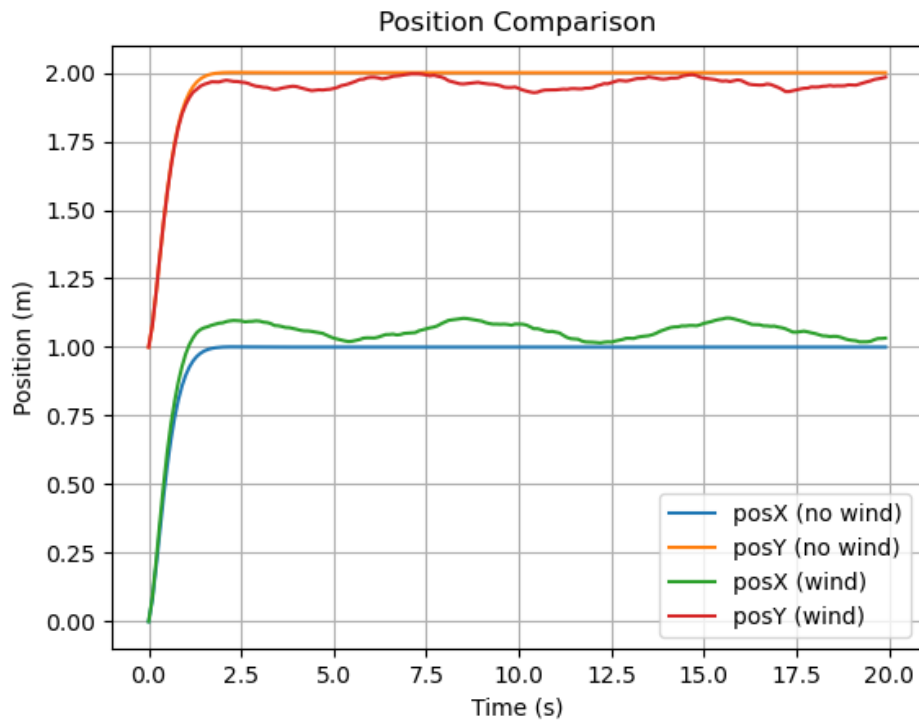


FIGURE 6 – Évolution des positions en fonction du temps (avec et sans perturbation)

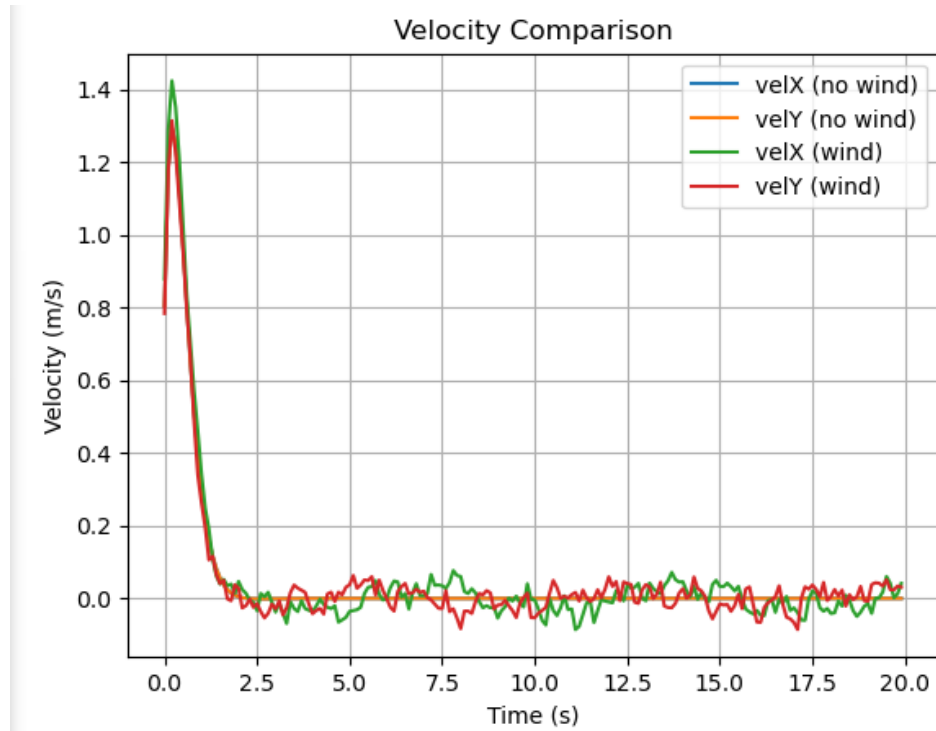


FIGURE 7 – Évolution des vitesses en fonction du temps (avec et sans perturbation)

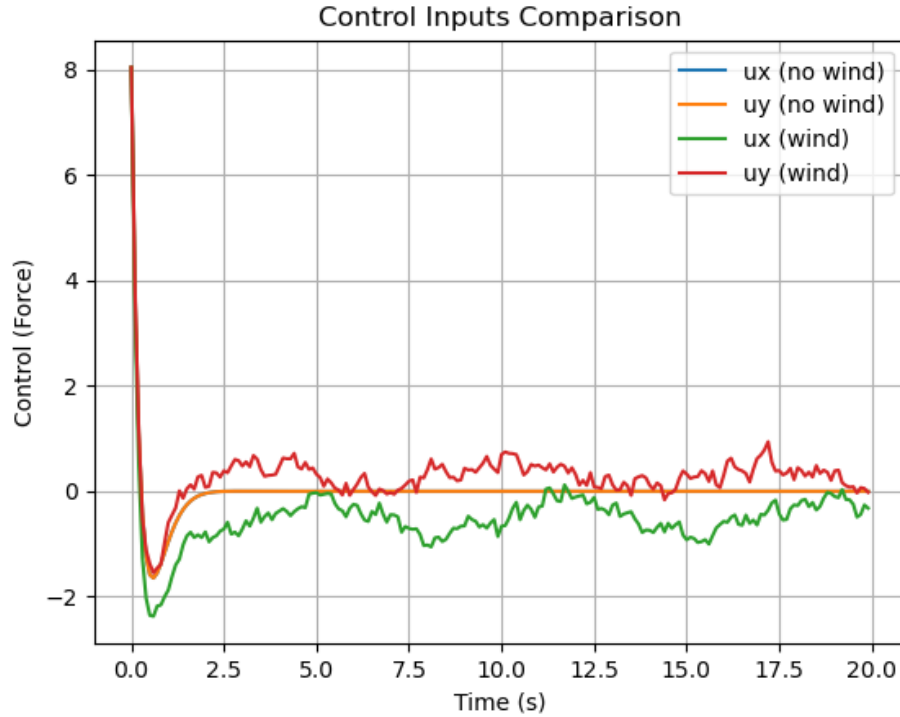


FIGURE 8 – Évolution des forces appliquées en fonction du temps (avec et sans perturbation)

4.3.2 Comportement du système simulé

L'analyse des figures 6, 7 et 8 met en évidence deux situations : le suivi de consigne en régime nominal et la réaction face à une perturbation.

Suivi nominal : En absence de perturbation, la position converge progressivement vers la cible. Les vitesses diminuent au cours du temps à mesure que le système se stabilise, traduisant un comportement amorti et sans oscillations marquées. Les forces appliquées sont plus importantes au début afin de corriger l'erreur initiale, puis diminuent lorsque le système s'approche de la consigne.

Réaction à la perturbation : Lorsque la perturbation est introduite, une déviation temporaire apparaît sur les positions et les vitesses. Le contrôleur réagit en ajustant immédiatement les forces u_x et u_y afin de compenser l'écart. Après cette phase transitoire, le système revient progressivement vers la position cible, et la convergence est rétablie malgré l'influence extérieure.

4.3.3 Interprétation générale

L'analyse des figures 6, 7 et 8 permet de confirmer le bon fonctionnement du contrôleur LQR.

Sur la figure 6, les courbes posX et posY convergent vers la cible dans les deux scénarios. En présence de perturbation (courbes “wind”), une déviation transitoire apparaît,

mais la convergence finale est maintenue.

La figure 7 montre que les vitesses $velX$ et $velY$ décroissent vers zéro après la phase transitoire, traduisant un comportement amorti. Les oscillations sont légèrement plus marquées en présence de vent, mais restent limitées.

Enfin, la figure 8 met en évidence l'augmentation temporaire des forces uX et uY dans le cas perturbé, illustrant la compensation active du contrôleur.

Ces observations confirment la capacité du régulateur à stabiliser le système tout en rejetant la perturbation.

5 Conclusion et Perspectives

Ce projet s'est attaché à combler le fossé entre l'ingénierie logicielle et la théorie du contrôle en développant une bibliothèque de contrôleurs réutilisables au sein du framework BIP. Partant d'une analyse approfondie de la littérature, nous avons réussi à concevoir, implémenter en C++ et valider trois approches fondamentales de la régulation : l'approche classique (SISO via le PID), l'approche prédictive sous contraintes (MPC), et l'approche multivariable coordonnée (MIMO via le LQR).

5.1 Bilan quantitatif et volume de développement

Afin d'estimer concrètement le volume de travail technique réalisé au cours de ce semestre, il convient de dresser un bilan quantitatif de notre développement. La création d'une bibliothèque générique et de son environnement de validation a nécessité un effort d'architecture et d'implémentation significatif, qui se traduit par les métriques suivantes :

- **Architecture logicielle (C++)** : L'ossature du projet repose sur **3** interfaces génériques (SISO, MIMO, MPC) et **5** classes concrètes pour la logique mathématique.
- **Intégration BIP** : Pour lier notre C++ au framework, nous avons développé **3** Wrappers spécifiques servant de façades fonctionnelles.
- **Volume de code** : Le projet totalise environ **7200** lignes de code réparties à travers ne nombreux fichiers (`.cpp` et `.hpp`) et scripts utilitaires.
- **Bancs de test** : **4** environnements de tests distincts ont été créés (environnements C++ purs et programmes BIP complets), permettant d'exécuter différents scénarios de validation (réponses à l'échelon, injections de bruit, perturbations fortes, limites de sécurité).
- **Gestion collaborative** : Le suivi rigoureux du projet se reflète par plus de **50** commits sur notre dépôt GitLab et plus d'une dizaine comptes-rendus de réunions documentés sur notre espace Notion.

Ces éléments tangibles illustrent la dimension globale du projet, qui ne s'est pas limitée à la simple écriture d'algorithmes, mais a englobé une véritable démarche d'ingénierie logicielle : de la conception architecturale à la validation, en passant par l'intégration continue.

5.2 Difficultés surmontées

La réalisation de ce projet ne s'est pas faite sans accroc. L'une des difficultés majeures a résidé dans la conception architecturale C++ permettant de lier nos algorithmes mathématiques complexes à la rigueur structurelle qu'impose BIP. La mise en place de Wrapper a été une étape clé pour résoudre les incompatibilités d'exécution liées à la syntaxe objet de notre bibliothèque et garantir une interface d'utilisation simple. Par ailleurs, la traduction de concepts théoriques continus (équations différentielles) vers une implémentation logicielle discrète a nécessité une grande rigueur, tout particulièrement lors de la gestion des matrices et de la résolution analytique des équations.

5.3 Ouvertures et travaux futurs

Bien que les résultats obtenus en simulation soient encourageants et valident notre approche, plusieurs pistes restent à explorer pour faire évoluer le projet.

Dans un premier temps, il sera essentiel de sortir du cadre de la simulation pour tester nos composants logiciels sur de véritables systèmes cyber-physiques, tels que les modules matériels Blinky Blocks. Passer sur du matériel physique permettra de confronter nos algorithmes aux contraintes réelles (latences, imprécisions de capteurs) qui sont difficiles à modéliser parfaitement.

Dans un second temps, l'enrichissement des bancs de validation est une priorité. Plus le nombre de tests sera important (en variant les scénarios, les intensités de perturbation et les modèles physiques), plus nous pourrons garantir la stabilité et la fiabilité de nos composants C++.

Par ailleurs, notre travail s'est concentré sur l'implémentation de trois contrôleurs spécifiques (PID, MPC et LQR), mais la littérature scientifique en propose une multitude d'autres. Ajouter de nouvelles stratégies de régulation permettrait d'offrir une bibliothèque BIP encore plus complète et adaptable aux besoins spécifiques des futurs développeurs.

En définitive, ce projet pose une première brique fonctionnelle pour le programme de recherche ADAPT. Il confirme que l'alliance entre l'ingénierie logicielle par composants (BIP) et la théorie du contrôle est une solution pertinente pour rendre les systèmes cyber-physiques plus fiables et robustes.

Références

- [1] Filieri, A., Maggio, M., Angelopoulos, K., d'Ippolito, N., et al. (2015). *Software Engineering Meets Control Theory*. Proceedings of the 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), pp. 71-82.
- [2] Goodwin, G. C., Graebe, S. F., & Salgado, M. E. (2001). *Control System Design*. Prentice Hall.
- [3] Laboratoire Verimag. *BIP Compiler Source Code Repository*. GitLab de l'Université Grenoble Alpes. Disponible sur : <https://gricad-gitlab.univ-grenoble-alpes.fr/verimag/bip/compiler>

Résumé

Ce rapport présente les travaux réalisés dans le cadre du projet MODELS UNDER CONTROL, dont l'objectif est d'intégrer les principes de la théorie du contrôle au sein du framework d'ingénierie logicielle BIP (Behavior, Interaction, Priority). Le but est de concevoir une bibliothèque de contrôleurs réutilisables en C++ pour stabiliser des Systèmes Cyber-Physiques (CPS) face à des perturbations environnementales.

À partir d'une étude de la littérature, trois stratégies de régulation discrètes ont été implémentées : une approche réactive classique (PID) pour les systèmes à une entrée et une sortie (SISO), une approche prédictive basée sur un modèle (MPC) garantissant le respect de contraintes de sécurité strictes, et un régulateur quadratique linéaire (LQR) pour la gestion coordonnée des systèmes multivariables (MIMO).

Pour intégrer ces algorithmes mathématiques dans l'architecture à composants de BIP, une conception logicielle modulaire basée des Wrapper a été mise en place.

Enfin, des bancs de tests ont permis de valider chaque contrôleur. Les simulations démontrent la robustesse du PID face au bruit, l'efficacité proactive du MPC (mode survie) sous perturbation forte, et la capacité du LQR à compenser les écarts sur un système couplé (modèle de drone).

Ce travail confirme la pertinence d'unifier l'ingénierie logicielle et l'automatique pour fiabiliser les systèmes critiques de demain.

Mots-clés : Systèmes Cyber-Physiques (CPS), Théorie du contrôle, Automatique, Framework BIP, C++, PID, Contrôle Prédictif (MPC), LQR, SISO, MIMO.

Abstract

This report presents the work conducted within the MODELS UNDER CONTROL project, which aims to integrate control theory principles into the BIP (Behavior, Interaction, Priority) software engineering framework. The goal is to design a library of reusable C++ controllers to stabilize Cyber-Physical Systems (CPS) against environmental perturbations.

Based on a literature review, three discrete control strategies were implemented : a classical reactive approach (PID) for Single-Input Single-Output (SISO) systems, a Model Predictive Control (MPC) approach ensuring strict compliance with safety constraints, and a Linear Quadratic Regulator (LQR) for the coordinated management of multivariable (MIMO) systems.

To integrate these mathematical algorithms into the BIP component-based architecture, a modular software design based on Wrappers was implemented.

Finally, test benches were used to validate each controller. The simulations demonstrate the robustness of the PID against noise, the proactive efficiency of the MPC ("survival mode") under strong perturbations, and the LQR's ability to compensate for deviations in a coupled system (drone model).

This work confirms the relevance of unifying software engineering and automatic control to ensure the reliability of tomorrow's critical systems.

Keywords : Cyber-Physical Systems (CPS), Control Theory, BIP framework, C++, PID, Model Predictive Control (MPC), LQR, SISO, MIMO.

Remerciements

Nous tenons tout d'abord à exprimer notre plus profonde gratitude à nos deux encadrantes de projet, Madame Olga KOUCHNARENKO et Madame Anna GALLONE. Leurs précieux conseils, leur disponibilité et leur bienveillance tout au long de ce semestre nous ont été d'une aide inestimable. Leurs retours constructifs et leur expertise nous ont permis d'appréhender sereinement la complexité de ce sujet et de surmonter les divers obstacles techniques et théoriques rencontrés.

Nos remerciements s'adressent également à Madame Sophie CERF. Son intervention et ses conseils avisés ont été un véritable atout pour notre équipe. Son regard expert nous a permis de nous aiguiller efficacement dans nos choix et d'orienter nos recherches dans la bonne direction.

Enfin, nous remercions l'ensemble de l'équipe pédagogique du Master ISL de l'Université Marie et Louis Pasteur pour la qualité de la formation dispensée, qui nous a fourni le socle de connaissances nécessaire à la réalisation de ce projet.

L'équipe du projet MODELS UNDER CONTROL